

TP : Communication, Sécurité et Contrôle d'un Robot Mobile sous ROS 2

Filière Robotique

Mai 2026

Table des matières

1	Partie 1 : Initialisation du Système (Vision par Moments & Contrôle P)	2
1.1	Idée générale	2
1.2	Travail demandé	2
2	Partie 2 : Optimisation Dynamique (Correcteur PD & Machine à États)	3
2.1	Idée générale	3
2.2	Travail demandé	3

Travaux Pratiques : Robotique Mobile & Autonomie

Partie II : Guidage par Vision Artificielle et Intégration Système sous ROS 2

Description du TP

L'objectif de ce TP est de mettre en œuvre un asservissement visuel rudimentaire. Dans une première partie, vous connecterez un nœud de traitement d'images OpenCV basé sur la méthode des **Moments géométriques** à un contrôleur proportionnel (P). Dans une seconde partie, vous ferez évoluer ce contrôleur vers un correcteur **Proportionnel-Dérivé (PD)** supervisé par une **Machine à États Finis (FSM)** afin de détecter et négocier les virages en épingle ainsi que les pertes de piste.

1 Partie 1 : Initialisation du Système (Vision par Moments & Contrôle P)

1.1 Idée générale

Le nœud de vision extrait une *Région d'Intérêt (ROI)* en bas de l'image pour anticiper la trajectoire immédiate du robot. À l'aide d'un seuillage et du calcul des moments d'ordre 0 et 1, on extrait la coordonnée horizontale C_x du centre de la ligne pour en déduire l'erreur de centrage par rapport à l'axe optique de la caméra.

1.2 Travail demandé

1. **Analyse et correction du binaire (Code Vision)** : Regardez la ligne du **code** de vision :

```
_, thresh = cv2.threshold(roi_gray, 200, 255, cv2.THRESH_BINARY)
```

Sachant que la piste est constituée de **lignes noires sur un sol clair**, expliquez pourquoi l'application de `cv2.THRESH_BINARY` va fausser le calcul du centre de la ligne opéré par `cv2.moments(thresh)`. Quelle modification devez-vous apporter au drapeau (*flag*) de la fonction pour corriger ce problème ?

2. **Compréhension mathématique des moments** : Rappelez la formule mathématique des moments M_{00} et M_{10} pour une image binarisée. Que représente physiquement le rapport $\frac{M_{10}}{M_{00}}$ calculé dans le code ?
3. **Mise en réseau ROS 2** : Lancez le nœud de **vision** corrigé ainsi que le nœud de commande proportionnel (suiveur2lignesProportionnel). Observez le comportement du robot. Pourquoi le déplacement linéaire est-il instable et saccadé lorsque le robot commence à osciller ?

2 Partie 2 : Optimisation Dynamique (Correcteur PD & Machine à États)

2.1 Idée générale

Pour lisser la trajectoire du robot et éviter qu'il ne rate la piste lors d'un virage brusque, nous implémentons un correcteur **Proportionnel-Dérivé (PD)** supervisé par une logique comportementale sous forme de **Machine à États Finis (FSM)** :

- **LIGNE_DROITE** : L'erreur de suivi est faible. La vitesse linéaire nominale est élevée et la correction angulaire est douce.
- **VIRAGE_BRUSQUE** : L'erreur dépasse un seuil critique. On réduit la vitesse linéaire par deux pour donner au robot le temps de pivoter sans glisser.
- **DEMI_TOUR** : Si la ligne sort brusquement du champ (perte de ligne), le robot applique un pivot sécurisé sur place pour retrouver la piste.

2.2 Travail demandé

1. **Analyse architecturale du code** : Examinez le script fourni. On remarque que les gains du correcteur (`gain_kp` et `gain_kd`) ont été définis localement dans le constructeur. Quel problème de portée (*scope*) cela va-t-il poser lors de l'exécution de la méthode `boucle()` ? Comment doit-on les déclarer correctement en Python ?
2. **Gestion du temps dynamique (dt)** : Pourquoi est-il indispensable d'utiliser une mesure de temps dynamique (`dt`) calculée entre deux frames plutôt qu'une valeur fixe théorique (comme 0.1 s) pour le calcul du terme Dérivé ?
3. **Rôle du terme Dérivé** : Lors de l'entrée brutale dans une courbe, comment réagit le terme Dérivé par rapport au terme Proportionnel ? En quoi la vitesse de variation de l'erreur (de/dt) aide-t-elle à anticiper le virage ?
4. **Implémentation de la FSM (Code à trous)** : Complétez la méthode `boucle()` du `script` pour intégrer la machine à états et le calcul de la commande de vitesse `msg.angular.z`. Testez votre nœud sur le simulateur ou le robot réel et validez le comportement.